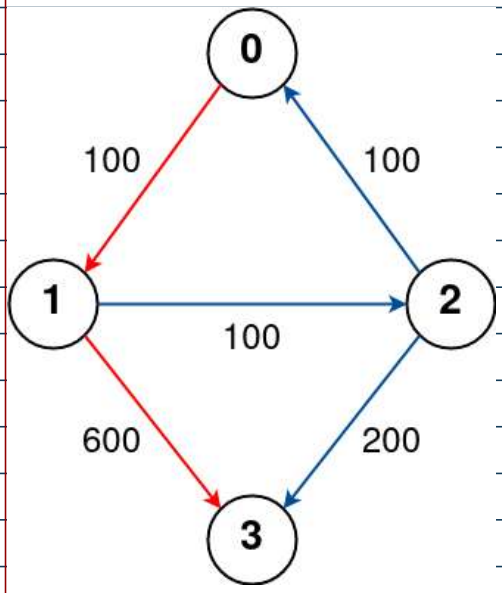


0 1 2

[[-, 100, 200, 400],
[200, -, 100, 300],
[100, 200, -, 200],
[-, -, -, -]]

100 + 200 < temp_prices[3]

price[u] price

(2) (2)—(3) (3)

n =
5

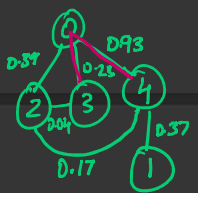
edges =
[[1,4],[2,4],[0,4],[0,3],[0,2],[2,3]]

succProb =
[0.37,0.17,0.93,0.23,0.39,0.04]

start_node =
3

end_node =
4

0.23 × 0.93 = 0.2139




```python
def maxProbability(self, n: int, edges: list[list[int]], succProb: list
[float], start_node: int, end_node: int) -> float:
    max_probs = [0]*n
    max_probs[start_node] = 1

    for _ in range(len(edges)):
        temp_mp = max_probs.copy()
        for i, e in enumerate(edges):
            u = e[0]
            v = e[1]
            if temp_mp[u] != 0:
                if temp_mp[u]*succProb[i] > temp_mp[v]:
                    temp_mp[v] = temp_mp[u]*succProb[i]
        max_probs = temp_mp

    return max_probs[end_node]
```

[0,0,0,0,0]
[0,0,0,1,0]

i=0, e=[1,4] i=3, e=[0,3]
i=1, e=[2,4] i=4, e=[0,2]
i=2, e=[0,4]

Actual output = 0

not considering the fact that it is undirected

[0,3] = [3,0]

Not considered, hence failing

```
        max_probs = [0]*n
        max_probs[start_node] = 1

        for _ in range(n):
            temp_mp = max_probs.copy()
            updated = False
            for i, e in enumerate(edges):
                u = e[0]
                v = e[1]
                # check u to v
                if temp_mp[u]*succProb[i] > temp_mp[v]:
                    temp_mp[v] = temp_mp[u]*succProb[i]
                    updated = True
                # check v to u
                if temp_mp[v]*succProb[i] > temp_mp[u]:
                    temp_mp[u] = temp_mp[v]*succProb[i]
                    updated = True
            max_probs = temp_mp

            if not updated:
                break

        return max_probs[end_node]
```

Time complexity
$O(V \cdot E)$

Like in Bellman Ford

Break is needed to "Early Stop"